

FINAL PROJECT

Giving Your Agents Persistent, Recoverable State, and Giving a Real User a Way to Reach Them

THE TASK

You're extending the same company, the same shared GitHub repo, the same database, and the same MCP server you have been building across the MCP Server Lab, the Memory & RAG Lab, and the Decomposition & Planning Lab. This your final project, and it changes two things about your system at once: how your agents hold state across interactions that are long, branching, or interrupted, and how a real user, not a grader reading your README, actually reaches any of your agents, and how a real admin actually runs the MCP server sitting behind them.

Every agent you've built so far assumes the run goes start to finish without anything happening that the agent can't quietly retry on its own. That assumption is false for a lot of real work in your company. You need an explicit graph of states and transitions, checkpointed as you go, so a run can pause, wait, fail, get inspected by a human, and resume from exactly where it left off instead of from the top.

You are not starting a new project and you are not throwing away the agents you already built. You are adding new state-graph-driven agents into the same `mcp_server/` and `db/`, sitting next to the memory/RAG agent and the decomposition/planning agent, and, for the first time in this course, building the actual product surface around all of them: a website or a platform where a real end user talks to whichever of your agents they need, and where a real admin manages what those agents can see and do.

THE STATE GRAPH & HUMAN-IN-THE-LOOP CONCERNS

A DAG, the kind your planning agent built, is acyclic and finite on purpose: it starts, it runs to a topological end, and it's done. A state graph is a different shape entirely: it can loop back to a state it's already visited, it can sit in a waiting state indefinitely until something outside the model happens, and it needs a persisted checkpoint at every meaningful transition, not just an execution log written after the fact. Your team must implement every concern below, and each one needs a genuine reason to exist in your system, not a demo built to check a box.

- **Three genuinely stateful problems, not one:** find three real, recurring requests in your company that each only make sense as a state graph. Each one needs interaction that genuinely spans more than one turn or one sitting, not just several tool calls back-to-back; a real branch that depends on something outside the model's control, a human's approval, a reply that may never come, an external system's response; and a real failure mode a single retry cannot fix. None of the three can be a re-skin of the scheduling problem from the Decomposition & Planning Lab or the retrieval problem from the Memory & RAG Lab, they must be a **genuinely new agent scope**. The three problems can live in one agent with three graphs, or three closely related agents, as long as you can justify the split.
- **Two of the four LLM-call additions, per graph:** for each of your three state graphs, build in two of the following inside its nodes: task decomposition, Tree of Thoughts or LATS, constrained ReAct, or a RAG architecture. The two additions do not have to be the same pair across all three graphs, pick whichever pair genuinely fits each problem's shape, and be able to say in your README why that node needed that technique and not one of the other three.
- **Human-in-the-loop escalation, routed to a real admin:** implement an explicit HITL node type in your graph. Define, in writing, the conditions under which a node must not let the agent decide alone, an amount above a threshold, an action that contradicts a stated policy, a confidence score below a bar you can defend.

When one of those conditions fires, the graph pauses, persists its full state, and opens a task for an admin, delivered through the platform you build, not a console print statement and not a side channel outside the product. The graph resumes only after the admin acts through the platform's UI, and the resumed run must pick up the admin's decision, not just proceed as if nothing happened.

- **A ticket-like system for failure and recovery:** every state graph needs a clean way to fail mid-node, with the run's checkpointed state persisted at the moment of failure, surfaced on the platform as a ticket with a real status (open, investigating, resolved), inspectable by a person, and resumable exactly from that checkpoint once resolved, not restarted from the beginning. A failure ticket is not the same code path as a HITL pause: HITL is an expected pause for a decision the agent isn't allowed to make on its own; a ticket is unplanned, a tool call errored, a schema validation failed, the model returned something the graph can't act on. Your system needs to demonstrate both paths, and a grader needs to be able to tell them apart in your code.
- **Checkpointing as a first-class citizen, not a log file:** your graph must write its state to durable storage after every meaningful transition, not only at the end of a run and not only when something goes wrong. Prove it: kill the process mid-run on purpose, restart it, and show the run resuming from its last checkpoint with no re-execution of steps that already completed and no loss of the state that was already collected.

If your project only demonstrates concerns that would work identically as a for-loop wrapped in a try/except, you haven't found a problem that genuinely needs a state graph. Go back and pick requests with a real wait, a real human decision, and a real cost to losing progress on failure.

THE PLATFORM: WHERE ADMINS AND USERS ACTUALLY MEET YOUR AGENTS

Every agent you've built across this course so far has been reachable only through a script, a notebook, or a grader running your code directly. That changes here. You must build a website or a platform for your chatbot system, an AI-generated site or an existing template is fine, either is acceptable, what matters is that it actually works end to end against your live MCP server and database, not a static mockup.

On the admin side,

- The platform must let an admin see every agent connected to the MCP server, and add or remove the tools available to each one. This requires your `mcp_server/` to actually support registering and de-registering tools at runtime or near-runtime, driven from the platform, not edited by hand and redeployed. This is also where your MCP server gets corrected: whatever your prior grading flagged as wrong or incomplete in the MCP Server Lab has to be genuinely fixed here, since it is now load-bearing for a real product feature.
- The platform must let an admin add and remove documents from the RAG system your Memory & RAG agent uses, with the change reflected in what that agent retrieves on the next query, not just uploaded to storage and ignored.
- The platform is also where HITL tasks and failure tickets from the concerns above actually surface and get resolved. An admin should be able to open a pending HITL request or an open ticket, see the graph's persisted state at the point it paused or failed, act on it, and watch the underlying run resume.

On the user side,

- a real end user must be able to switch between agents, whichever of your state-graph agents, your memory/RAG agent, and your decomposition/planning agent are live, and chat with whichever one they need, through whatever interface pattern your team judges viable, a sidebar switcher, separate chat tabs, a single

unified thread that routes under the hood, your call, as long as it is genuinely usable and not a single hardcoded conversation.

CORRECT WHAT CAME BEFORE

You are one team on one repository for the entire course, and grading for this project covers the whole thing, not just the new folders you add this time. Before you build anything new, go back through the feedback and the rubric gaps from the MCP Server Lab, the Memory & RAG Lab, and the Decomposition & Planning Lab, and fix them for real. A state-graph agent that calls a broken or incomplete MCP tool, or a platform that manages a RAG store your retrieval agent doesn't actually use correctly, does not get credit for being new work sitting next to old, unfixed work. Treat this as a chance to bring the entire system, every agent, the server, the database, and now the platform, up to one consistent, working standard.

A WORKED EXAMPLE, READ THIS, THEN DO SOMETHING DIFFERENT WITH YOUR OWN COMPANY

This shows the shape of the exercise on top of your MCP Server Lab, Memory & RAG Lab, and Decomposition & Planning Lab examples, not the problems to copy. Pick your own three problems from your own company's real workflows.

Picture “Larkspur Veterinary Clinics” again. The memory/RAG agent handles front-desk triage and clinical-policy questions. The scheduling agent from the Decomposition & Planning Lab reshuffles the day's appointment board. Neither of those holds state across days, waits on an outside reply, or needs a human to sign off mid-run, so neither is a candidate here. The team instead builds three new state-graph problems, each genuinely different in shape:

State-graph problem	Why it can't be a single pass	Two additions used
Multi-visit chronic-condition treatment coordination	Spans weeks, waits on lab results that arrive on their own schedule, and needs a vet's sign-off before the plan changes.	Task decomposition (build the visit sequence) + RAG (pull the clinic's chronic-care protocols)
Pet-insurance claim submission and appeal	Waits on an external insurer's response, can be rejected and need a reasoned appeal, and a wrong resubmission wastes a real claim window.	Constrained ReAct (fill and submit the correct insurer forms) + Tree of Thoughts (choose which appeal argument to lead with)
Emergency intake triage handoff	An irreversible action (sedation, admission) must not be taken by the agent alone, and a wrong triage order costs real time in an emergency.	LATS (search over triage orderings against a real severity check) + constrained ReAct (execute only whitelisted intake actions)

For the treatment-coordination graph, a state named `awaiting_lab_results` genuinely waits, sometimes for days, and only transitions once a lab-integration webhook lands; the vet's sign-off before changing the plan is the HITL node, delivered as a task on the platform, and a lab result that never arrives within the expected window opens a ticket rather than looping forever. For the claims graph, a rejected submission triggers Tree of Thoughts over a small set of appeal strategies before constrained ReAct fills and files the correct follow-up forms through the MCP server's real claims-write tool; a malformed insurer response that the graph can't parse becomes a ticket, not a silent failure. For the intake-triage graph, LATS searches over candidate orderings scored by a real acuity check, not the model's own opinion of urgency, and any action that would sedate or admit an animal is a HITL node that pages the on-call vet through the platform before the graph is allowed to proceed.

None of these three would work as a straight-line script: each one has a real wait, a real branch that depends on something outside the model, and a real cost to losing progress if the process dies mid-run. That's the bar your own three problems need to clear.

WHAT TO BUILD AND SUBMIT

- **An updated README:** the three state-graph problems you found, why each one is real, which agent owns each, how every concern above shows up in your solution, and what you fixed from prior grading.
- **A `state_graph/` (or similarly named) folder:** your three graphs, wired into your real MCP tools and database, with the HITL node type and the ticket/failure-recovery path both implemented and locatable, plus the checkpointing layer that makes crash-and-resume possible.
- **A `platform/` folder:** the website's source, covering the admin surface (tool add/remove per agent, RAG document add/remove, HITL and ticket resolution) and the user surface (agent switching and chat), wired against your live backend.
- **Any changes to `mcp_server/` or `agent/`** needed to support runtime tool management from the admin panel and to stand up the new state-graph agents, visibly reusing the existing server and database rather than standing up a parallel one.
- **Locatable concerns:** inside `state_graph/`, a grader should be able to find the graph and cycle definitions, the checkpointing calls, the HITL node, and the ticket/failure path without reading the whole file.
- **Demo evidence:** a recording or transcript showing a run pausing on a genuine HITL condition and being resolved by an admin through the platform, a run failing mid-node and becoming a ticket that gets resolved and resumed from its checkpoint, and a process being killed and restarted mid-run to show recovery.
- **Commit history and issues** showing real contributions from every team member, including commits that fix flagged faults in the prior three labs.

RESOURCES

- [LangGraph Persistence & Checkpointing Guide](#)
- [LangGraph Docs.](#)
- [LangGraph Dynamic Interrupts & Time Travel](#)
- [LangChain Human-in-the-Loop Middleware & Policies](#)
- [Vercel AI SDK Chat Template \(Next.js\)](#)

A FEW GUARDRAILS

- Pick problems where waiting, a human decision, or a failure genuinely changes the outcome. If your graph would produce the same result run straight through with no pauses, you haven't found a state-graph problem.
- Don't fake human-in-the-loop by auto-approving in code somewhere the grader isn't likely to read. The pause has to be real, the resume has to depend on a real admin action taken through the platform.
- Don't fake tickets. A ticket has to come from an actual failure your system detected, not a manually inserted database row created for the demo.

- Checkpointing must survive an actual process restart, not just a pause-and-resume within the same running process. Test it by actually killing the process.
- Tool add/remove from the admin panel must actually reach the live MCP server. A UI toggle that doesn't change what tools the agent can call earns no credit for this concern.
- Don't rebuild your existing MCP server, database, or prior agents from scratch. This project is graded on genuine extension and correction of existing work, in every direction.
- Never commit an API key or database credential. Confirm your .env file is still in .gitignore, and check the platform's own configuration for the same mistake, a website is a new place to leak a secret.

WORKING AS A TEAM: ISSUES AND MODULAR WORK

Every GitHub Issue is graded on its rationale, not its existence. An issue that says “add HITL” earns nothing. One that states the problem, the constraint, and the acceptance criteria does.

Split the work across the three state-graph problems (one or more owners each), the platform's admin surface, the platform's user surface, the ticket/failure-recovery system, and the corrections carried over from the prior three labs. Each concern gets its own owner before any code is written.

Write issues with real rationale, for example “the office manager currently has no way to see why a claim graph stalled for three days, and a stalled claim past the insurer's window loses the appeal entirely” rather than “add ticket system.” Name the constraint that makes the concern genuinely necessary in your system, and write acceptance criteria a teammate could check without asking what you meant.

One owner per issue, referenced from the pull request that resolves it (“Closes #-”), with a short closing comment explaining what changed. An issue opened and closed with no linked commit or discussion won't count as attributable teamwork.

PRESENTATION

Presentation is graded, and every member of every team needs to present. Each team gets ten minutes total, including discussion, so plan your slides and your live demo to fit inside that window with time left for questions (3 minutes minimum).

- Come with well-prepared slides and a live demo of the actual website, not screenshots, walking through both the admin surface and the user surface against your real running system.
- Do not spend your time explaining what a state graph, HITL, or a ticket system is. That's assumed knowledge going into the room. Spend your time explaining the three specific problems you found in your company and exactly how your three agents solve them: what state they hold, what makes each one wait or branch, and what happens when a human has to step in.
- Be ready to answer discussion questions on the spot, about why a given problem needed a state graph instead of a simpler pattern, why you paired the two LLM-call additions you chose for a given graph, or what a specific ticket in your system actually catches.

GRADING RUBRIC

Your team deliverable is scored out of 100 fixed points, covering this project on top of the whole repository. Every row is binary, full points only when its criteria are genuinely met, not partially. Your final individual grade is 70% this team score plus 30% an individual contribution score built from Git history and issue rationale.

Category	Points	What's assessed
Problem framing and suitability	10	Three genuinely stateful problems on a new agent (or bounded family of agents), distinct from the memory/RAG agent and the decomposition/planning agent; each one truly needs cycles, pauses, and recovery rather than a single linear run.
Extending and correcting the existing system	8	mcp_server/, db/, and every prior agent are visibly reused, not duplicated; faults flagged in the MCP Server Lab, the Memory & RAG Lab, and the Decomposition & Planning Lab are genuinely fixed, since grading now covers the whole repo.
State graph architecture and checkpointing	15	All three problems correctly modeled as state graphs with real cycles and branches, state persisted to durable storage after each meaningful transition, and a demonstrated crash-and-resume from a mid-run checkpoint.
Two LLM-call additions per graph	15	Each of the three graphs correctly integrates two of task decomposition, Tree of Thoughts/LATS, constrained ReAct, and RAG, placed inside specific nodes for a reason tied to that node's job, not bolted on for coverage.
Human-in-the-loop escalation	12	A real HITL node type that pauses execution, persists full state, and opens a task on the platform for a real admin to act on, with the graph only resuming after that action is taken through the platform's UI.
Ticket system for failure and recovery	15	Unplanned mid-node failures open a real, persisted ticket distinct from a HITL pause, inspectable and resolvable by a user through the platform, with the run resuming from its last checkpoint, not restarting from scratch.
The platform	10	A working website where admins can add and remove MCP tools per agent and add and remove RAG documents, and where end users can switch between agents and chat with each; tool changes visibly reach the live MCP server.
Repository usability and safety	5	Clear organization, demo evidence covering every concern above, reproducible setup instructions, no exposed secrets or credentials.
Teamwork and issue rationale	10	Issues with genuine rationale, single ownership, and traceable resolution through linked pull requests, across every concern in this project and the fixes carried over from prior ones.

Total	100	
--------------	------------	--